

Lab 18 - PersistentVolumes, Claims and StorageClasses

Field	Value
Lab id	Lab 18
Course	TGS-2023039343 Kubernetes and Cloud Native Associate (KCNA) Training v6.0
Day / Topic	Day 4 Storage
Duration	55 minutes
Namespace	kcna-lab18
Maps to	LO4 Prepare a technical blueprint for a Kubernetes-based solution for security and storage A4 Prepare a technical blueprint for a solution in a given area K6 Technical blueprint design and construction process

1. Objective

By the end of this lab you will be able to:

1. Explain the division of labour between a PersistentVolume (cluster-scoped supply, an administrator's concern) and a PersistentVolumeClaim (namespaced demand, a developer's concern).
2. Bind a hand-written static PV to a PVC using storageClassName and claimRef, and state why the bound capacity is 1Gi when the claim asked for 512Mi.
3. Prove persistence by writing data from one Pod, deleting that Pod, and reading the data back from a different Pod.
4. Choose correctly between the access modes ReadWriteOnce, ReadOnlyMany, ReadWriteMany and ReadWriteOncePod, and state which of them a single-node kind cluster can actually deliver.
5. Contrast the reclaim policies Retain and Delete, and recognise the Released phase.
6. Provision dynamically through kind's default standard StorageClass, and correctly identify a Pending PVC caused by volumeBindingMode: WaitForFirstConsumer as expected behaviour rather than a fault.
7. Diagnose a genuinely broken Pending PVC - a StorageClass name typo - from kubectl describe pvc events.

2. Prerequisites

- * A single-node kind cluster on Kubernetes v1.30 or later.
- * Lab 17 completed. This lab is the answer to the failure you diagnosed there.
- * Terminal at this lab folder:

```
cd courseware/labs/lab-18-pv-pvc-storageclass
pwd
```

```
.../courseware/labs/lab-18-pv-pvc-storageclass
```

What kind gives you out of the box

Before you write anything, find out what storage this cluster already has. This matters: the whole lab depends on it.

```
kubectl get storageclass
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION standard (default) 41m	AGE rancher.io/local-path	Delete	WaitForFirstConsumer false

Read every column:

Column	Value	What it means for this lab
NAME	standard (default)	The (default) suffix means a PVC that omits storageClassName gets this class automatically.
PROVISIONER	rancher.io/local-path	kind ships the local-path-provisioner. It is <i>*not*</i> a CSI driver - it is a simple in-cluster controller that creates a directory under /opt/local-path-provisioner on the node and wraps it in a PV.
RECLAIMPOLICY	Delete	Deleting a PVC from this class destroys the underlying directory and the PV with it.
VOLUMEBINDINGMODE	WaitForFirstConsumer	Claims stay Pending until a Pod consumes them. You will see this in Step 9 and it is <i>*correct*</i> .
ALLOWVOLUMEEXPANSION	false	You cannot grow a local-path volume by editing the PVC.

Confirm the provisioner is actually running:

```
kubectl -n local-path-storage get pods
```

NAME	READY	STATUS	RESTARTS	AGE
local-path-provisioner-7dc846544d-mrxsq	1/1	Running	0	41m

Honesty note about this cluster. On a real production cluster the provisioner would be a CSI driver - ebs.csi.aws.com, pd.csi.storage.gke.io, disk.csi.azure.com, rook-ceph.rbd.csi.ceph.com - running as a controller Deployment plus a node DaemonSet, and the volumes would be network-attached block devices or filesystems that survive node loss. kind's local-path-provisioner produces node-local directories. Everything you learn about the PV/PVC/StorageClass API transfers exactly; the durability guarantees do not. Be explicit about that distinction when you write a storage blueprint.

3. Scenario

Kallang Freight's TrackLane platform has a statutory problem. Singapore Customs requires sealed export manifests to be retained for seven years. Right now those manifests are written into a container filesystem - the same design you proved fatal in Lab 17 - and the compliance team has flagged it as an audit finding.

The platform team has produced a retention policy (data/retention-policy.csv) that classifies every TrackLane dataset into gold, silver and bronze tiers with a required reclaim policy for each. Your job in this lab is to implement the two extremes of that policy:

* customs-manifest-archive - *gold tier, Retain*. Storage supplied deliberately by an administrator, bound to exactly one claim, and surviving the deletion of that claim.

* tracking-render-cache - *bronze tier, Delete*. Storage summoned on demand from a StorageClass, thrown away with its claim.

Then you will be handed a real ticket: **"the archive PVC has been Pending for twenty minutes"**.

4. Step-by-step procedure

Step 1 - Namespace and datasets

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab18 created
```

Look at the two datasets you will use:

```
head -3 data/manifest-archive.csv
echo "rows: $(( $(wc -l < data/manifest-archive.csv) - 1 ))"
```

```
archive_id,sealed_utc,shipment_id,consignee,hs_code,declared_value_sgd,customs_ref,depot
ARC-2026-000101,2026-08-24T01:05:00Z,KF-SG-100001,Pahang Agri Bhd,0803.10,18450.00,SG-EXP-
884201,depot-a
ARC-2026-000102,2026-08-24T02:41:00Z,KF-SG-100002,Sentosa Marine Supply,8481.80,127300.00,SG-
EXP-884202,depot-b
rows: 30
```

```
column -s, -t data/retention-policy.csv | cut -c1-96
```

dataset	tier	access_mode	reclaim_policy	retention_days	storage_class
customs-manifest-archive	gold	ReadWriteOnce	Retain	2555	tracklane-
manual					
depot-ledger-state	gold	ReadWriteOnce	Retain	2555	tracklane-
retain					
tracking-render-cache	bronze	ReadWriteOnce	Delete	1	standard
lane-etl-scratch	bronze	ReadWriteOnce	Delete	1	standard
ops-report-snapshots	silver	ReadWriteOnce	Delete	90	standard
partner-edi-inbox	silver	ReadWriteOnce	Retain	365	tracklane-
retain					

If column is unavailable, cat data/retention-policy.csv shows the same content unaligned.

Load both into the cluster:

```
kubectl -n kcna-lab18 create configmap tracklane-archive-seed \
  --from-file=manifest-archive.csv=data/manifest-archive.csv
kubectl -n kcna-lab18 create configmap tracklane-retention-policy \
  --from-file=retention-policy.csv=data/retention-policy.csv
```

```
configmap/tracklane-archive-seed created
configmap/tracklane-retention-policy created
```

Step 2 - Understand access modes before you choose one

This is examinable and widely misunderstood. An access mode is a property of the volume, requested by the claim, and enforced by the provisioner - Kubernetes itself does not magically make a volume shareable.

Mode	Short	Meaning	Available on a single-node kind cluster?
ReadWriteOnce	RWO	Mountable read-write by Pods on one node. Multiple Pods on that *same* node may mount it.	Yes. local-path supports this.
ReadOnlyMany	ROX	Mountable read-only by Pods on many nodes simultaneously.	Not meaningfully - there is only one node, and local-path does not advertise ROX.
ReadWriteMany	RWX	Mountable read-write by Pods on many nodes simultaneously. Needs a shared filesystem: NFS, CephFS, EFS, Azure Files.	No. No block-device-backed driver can offer this, and local-path does not.
ReadWriteOncePod	RWOP	Mountable read-write by exactly one Pod in the whole cluster. Stable since v1.29; requires a CSI driver.	No. RWOP is enforced by the CSI layer; local-path is not a CSI driver.

The trap. ReadWriteOnce says *one node*, not *one Pod*. Two Pods scheduled to the same node can both mount an RWO volume read-write and corrupt each other. ReadWriteOncePod was added in v1.22 (stable v1.29) precisely to close that gap. If you need single-writer semantics, RWO is not enough.

Everything in this lab therefore uses RWO, which is the honest answer for this cluster.

Step 3 - Create the static PersistentVolume

Read the manifest, then apply it:

```
kubectl apply -f manifests/01-static-pv.yaml
```

```
persistentvolume/tracklane-archive-pv created
```

```
kubectl get pv tracklane-archive-pv
```

NAME	STORAGECLASS	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
		VOLUMEATTRIBUTESCLASS	REASON	AGE		
tracklane-archive-pv	1Gi	RWO	Retain		Available	kcna-lab18/
archive-claim	tracklane-manual	<unset>			6s	

Two things worth stopping on:

* STATUS: Available - the PV exists and is unbound.

* CLAIM: kcna-lab18/archive-claim is already populated even though no such claim exists yet. That is the claimRef you wrote: the PV is *reserved*. Any other PVC that tries to take it will be refused.

VOLUMEATTRIBUTESCLASS appears from Kubernetes v1.29 onward (the VolumeAttributesClass alpha/beta feature for mutable volume QoS). <unset> is normal. On older clients the column is absent.

Confirm it is cluster-scoped - it has no namespace:

```
kubectl get pv tracklane-archive-pv -o jsonpath='{.metadata.namespace}{"[end]"}{"\n"}'
```

```
[end]
```

Step 4 - Create the claim and watch it bind

```
kubectl apply -f manifests/02-static-pvc.yaml
```

```
persistentvolumeclaim/archive-claim created
```

```
kubectl -n kcna-lab18 get pvc archive-claim
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
VOLUMEATTRIBUTESCLASS	AGE				
archive-claim	Bound	tracklane-archive-pv	1Gi	RWO	tracklane-manual
<unset>	4s				

Bound, immediately. Note this carefully, because it contrasts with Step 9:

* This PV/PVC pair uses storageClassName: tracklane-manual, which is a class name no StorageClass object owns. There is no provisioner and therefore no volumeBindingMode, so the PersistentVolume controller binds as soon as it finds a match. Binding is *immediate*.

* CAPACITY reads 1Gi, not the 512Mi you requested. Binding is "at least as large as requested"; the Pod receives the entire PV. In a cloud, a 512Mi claim that binds to a hand-made 1Gi volume bills you for 1Gi.

Look at it from the PV's side:

```
kubectl get pv tracklane-archive-pv -o custom-columns=\
'NAME:.metadata.name,STATUS:.status.phase,CLAIM:.spec.claimRef.name,UID:.spec.claimRef.uid'
```

NAME	STATUS	CLAIM	UID
tracklane-archive-pv	Bound	archive-claim	1c9f4a02-6e5b-4d81-93a7-0b2e5f7c4d16

The controller has filled in the claim's UID. This is what makes binding one-to-one and permanent: even if you delete the PVC and recreate one with the identical name, the UID differs and the PV will not re-bind to it. That behaviour is the subject of Step 12.

Step 5 - Seed the archive into the volume

```
kubectl apply -f manifests/03-archive-seeder.yaml
```

```
pod/archive-seeder created
```

```
kubectl -n kcna-lab18 wait --for=jsonpath='{.status.phase}'=Succeeded \
  pod/archive-seeder --timeout=120s
```

```
pod/archive-seeder condition met
```

```
kubectl -n kcna-lab18 logs archive-seeder
```

```
wrote 30 archive rows to the PersistentVolume
sealed_by=archive-seeder
sealed_utc=2026-09-05T05:02:41Z
archive_rows=30
retention_class=gold
```

30 rows - matching data/manifest-archive.csv. The dataset has travelled: file on disk → ConfigMap → volume mount → PersistentVolume.

Now delete the writer. This is the important move:

```
kubectl -n kcna-lab18 delete pod archive-seeder --wait=true
```

```
pod "archive-seeder" deleted
```

```
kubectl -n kcna-lab18 get pods
```

```
No resources found in kcna-lab18 namespace.
```

No Pod in the namespace. The claim, however, is untouched:

```
kubectl -n kcna-lab18 get pvc archive-claim
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
VOLUMEATTRIBUTESCLASS		AGE			
archive-claim	Bound	tracklane-archive-pv	1Gi	RWO	tracklane-manual
<unset>		3m12s			

Step 6 - Read the archive back from a different Pod

```
kubectl apply -f manifests/04-archive-reader.yaml
kubectl -n kcna-lab18 wait --for=condition=Ready pod/archive-auditor --timeout=120s
```

```
pod/archive-auditor created
pod/archive-auditor condition met
```

```
kubectl -n kcna-lab18 logs archive-auditor
```

```

--- provenance written by the (now deleted) seeder Pod ---
sealed_by=archive-seeder
sealed_utc=2026-09-05T05:02:41Z
archive_rows=30
retention_class=gold
--- archive row count as read back ---
rows_read=30
--- can this auditor modify the archive? ---
read-only mount refused the write (expected)

```

This is the result the whole lab exists for. `sealed_by=archive-seeder` was written by a Pod that no longer exists, and `rows_read=30` matches the source dataset byte for byte. Compare with Lab 17, where the identical experiment on an `emptyDir` produced a fresh build timestamp every time.

Step 7 - Where do the bytes actually live? (kind only, optional)

```
kubectl get pv tracklane-archive-pv -o jsonpath='{.spec.hostPath.path}{"\n"}'
```

```
/tmp/tracklane-archive
```

```
docker exec kcna-control-plane ls -l /tmp/tracklane-archive
```

```

total 8
-rw-r--r-- 1 root root   96 Sep  5 05:02 PROVENANCE
-rw-r--r-- 1 root root 3341 Sep  5 05:02 manifest-archive.csv

```

Substitute your kind node container name from `docker ps --format '{{.Names}}'`. Read only - never modify anything under a live PV from the node side. And note the honesty point again: this is `/tmp` *inside the kind node container*, not on your laptop. `kind delete cluster` destroys it. A Retain reclaim policy protects you from the Kubernetes control plane, not from losing the node.

Step 8 - Register the gold-tier StorageClass

`data/retention-policy.csv` says `depot-ledger-state` and `partner-edi-inbox` need dynamic provisioning with Retain. `kind`'s standard class uses Delete, so you need a second class.

```
kubectl apply -f manifests/05-storageclass-retain.yaml
```

```
storageclass.storage.k8s.io/tracklane-retain created
```

```
kubectl get storageclass
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	
ALLOWVOLUMEEXPANSION	AGE			
standard (default)	rancher.io/local-path	Delete	WaitForFirstConsumer	false
52m				
tracklane-retain	rancher.io/local-path	Retain	WaitForFirstConsumer	false
5s				

Same provisioner, different lifecycle. That is the whole idea of a `StorageClass`: it is a named profile over a driver.

```
kubectl describe storageclass tracklane-retain | head -8
```

```
Name:                tracklane-retain
IsDefaultClass:      No
Annotations:         storageclass.kubernetes.io/is-default-class=false
Provisioner:         rancher.io/local-path
Parameters:          <none>
AllowVolumeExpansion: False
MountOptions:        <none>
ReclaimPolicy:       Retain
```

A word on CSI. In-tree storage drivers were removed from the Kubernetes codebase over the v1.21-v1.31 cycle; every cloud volume type is now an out-of-tree CSI driver. CSI ("Container Storage Interface") is a gRPC contract with three services - Identity, Controller and Node - that a vendor implements once and every orchestrator can consume. `kubectl get csidrivers` lists the drivers registered on your cluster. On a stock kind cluster the list is empty, because local-path-provisioner predates CSI and is a plain controller:

```
>
```

```
```bash
```

```
kubectl get csidrivers
```

```
```
```

```
>
```

```
```
```

```
No resources found
```

```
```
```

```
>
```

That empty output is itself worth showing learners: kind is teaching you the API, not the production storage stack.

```
---
```

Step 9 - Dynamic provisioning, and the Pending you should NOT panic about

```
kubectl apply -f manifests/06-dynamic-pvc.yaml
```

```
persistentvolumeclaim/render-cache created
```

```
kubectl -n kcna-lab18 get pvc render-cache
```

| NAME | STATUS | VOLUME | CAPACITY | ACCESS | MODES | STORAGECLASS |
|-----------------------|---------|--------|----------|--------|-------|--------------|
| VOLUMEATTRIBUTESCLASS | AGE | | | | | |
| render-cache | Pending | | | | | standard |
| 8s | | | | | | <unset> |

Pending with no volume. Do not debug this. Read the events:

```
kubectl -n kcna-lab18 describe pvc render-cache | tail -8
```

```

Used By:      <none>
Events:
  Type        Reason                Age    From                                Message
  ----        -
  Normal      WaitForFirstConsumer  3s (x2 over 10s)  persistentvolume-controller  waiting for
first consumer to be created before binding

```

Reason: WaitForFirstConsumer, Type: Normal (not Warning). The controller is telling you, in as many words, that it is deliberately holding off.

Why the behaviour exists. If the provisioner created the volume immediately it would have to guess a node or a zone. In a multi-zone cluster that guess is wrong two-thirds of the time, and you get a Pod that can never be scheduled because its volume is in ap-southeast-1a and its only free capacity is in 1b. WaitForFirstConsumer inverts the order: schedule the Pod first, *then* provision the volume where the Pod landed.

How to tell this apart from a real fault - memorise this table:

| Event Type | Event Reason | Verdict |
|----------------------|-------------------------------------|--|
| Normal | WaitForFirstConsumer | Expected. Create a Pod that uses the claim. |
| Normal | Provisioning / ExternalProvisioning | In progress. Wait, then check the provisioner Pod's logs. |
| Warning | ProvisioningFailed | Real fault. Read the message - bad class, quota, driver error. |
| *(no events at all)* | - | No matching PV and no provisioner is even looking. Usually a class name that no StorageClass owns. |

Step 10 - Schedule the first consumer

```
kubectl apply -f manifests/07-render-cache-deployment.yaml
```

```
deployment.apps/render-cache created
```

```
kubectl -n kcna-lab18 rollout status deployment/render-cache --timeout=180s
```

```

Waiting for deployment "render-cache" rollout to finish: 0 of 1 updated replicas are available.
...
deployment "render-cache" successfully rolled out

```

```
kubectl -n kcna-lab18 get pvc render-cache
```

| NAME | STATUS | VOLUME | | CAPACITY | ACCESS MODES |
|--------------|-----------------------|--|-------|----------|--------------|
| STORAGECLASS | VOLUMEATTRIBUTESCLASS | AGE | | | |
| render-cache | Bound | pvc-8b2e17d4-05c9-4f36-a1b7-e39d602c8a54 | 256Mi | RW0 | |
| standard | <unset> | 2m41s | | | |

Bound, and the volume name is pvc-<uuid> - machine-generated, because no human wrote this PV. Look at what the provisioner created:

```
DYNPV=$(kubectl -n kcna-lab18 get pvc render-cache -o jsonpath='{.spec.volumeName}')
kubectl get pv "$DYNPV"
```

| NAME | STORAGECLASS | CAPACITY | ACCESS MODES | RECLAIM POLICY | STATUS |
|--|--------------|-----------------------|--------------|----------------|--------|
| CLAIM | | VOLUMEATTRIBUTESCLASS | REASON | AGE | |
| pvc-8b2e17d4-05c9-4f36-a1b7-e39d602c8a54 | | 256Mi | RWO | Delete | Bound |
| kcna-lab18/render-cache | standard | <unset> | | 34s | |

Note RECLAIM POLICY: Delete, inherited from the standard class. Compare with tracklane-archive-pv, which is Retain.

```
kubectl -n kcna-lab18 logs deployment/render-cache
```

```
--- /cache/starts.log ---
start render-cache-6c9f84b57d-x2klp 2026-09-05T05:14:08Z
--- bronze-tier datasets from the retention policy ---
tracking-render-cache
lane-etl-scratch
```

The retention policy dataset is genuinely being read inside the Pod.

Now restart the Deployment and prove the dynamic volume persists too:

```
kubectl -n kcna-lab18 rollout restart deployment/render-cache
kubectl -n kcna-lab18 rollout status deployment/render-cache --timeout=180s
kubectl -n kcna-lab18 logs deployment/render-cache | head -4
```

```
deployment.apps/render-cache restarted
deployment "render-cache" successfully rolled out
--- /cache/starts.log ---
start render-cache-6c9f84b57d-x2klp 2026-09-05T05:14:08Z
start render-cache-7fb5d9c684-qn8vt 2026-09-05T05:15:52Z
```

Two lines. The second Pod appended to a file the first Pod created. The Recreate strategy mattered here: with RollingUpdate, the new Pod would have tried to mount an RWO volume still held by the old Pod.

Step 11 - Full picture

```
kubectl -n kcna-lab18 get pvc,pod -o wide
```

| NAME | CAPACITY | ACCESS MODES | STORAGECLASS | STATUS | VOLUME | AGE |
|-------------------------------------|----------|------------------|--------------|--------|--|-----|
| persistentvolumeclaim/archive-claim | | | | Bound | tracklane-archive-pv | 16i |
| | RWO | tracklane-manual | | 14m | | |
| persistentvolumeclaim/render-cache | | | | Bound | pvc-8b2e17d4-05c9-4f36-a1b7-e39d602c8a54 | |
| | 256Mi | RWO | standard | | 5m | |

| NAME | READY | STATUS | RESTARTS | AGE | IP | NODE |
|-----------------------------------|-------|---------|----------|-----|-------------|-------|
| pod/archive-auditor | 1/1 | Running | 0 | 9m | 10.244.0.44 | kcna- |
| control-plane | | | | | | |
| pod/render-cache-7fb5d9c684-qn8vt | 1/1 | Running | 0 | 2m | 10.244.0.46 | kcna- |
| control-plane | | | | | | |

One claim satisfied by an administrator's hand-made volume, one satisfied by a provisioner - and the Pod specs are identical in shape. That is the abstraction paying off: the workload author writes persistentVolumeClaim:

{claimName: ...} and never learns which mechanism supplied the bytes.

Step 12 - Reclaim policy in action: Retain and the Released phase

Delete the auditor and then the gold-tier claim:

```
kubectl -n kcna-lab18 delete pod archive-auditor --wait=true
kubectl -n kcna-lab18 delete pvc archive-claim --wait=true
```

```
pod "archive-auditor" deleted
persistentvolumeclaim "archive-claim" deleted
```

```
kubectl get pv tracklane-archive-pv
```

| NAME | STORAGECLASS | CAPACITY | ACCESS MODES | RECLAIM POLICY | STATUS | CLAIM |
|----------------------|------------------|----------|-----------------------|----------------|----------|-------------|
| | | | VOLUMEATTRIBUTESCLASS | REASON | AGE | |
| tracklane-archive-pv | | 1Gi | RWO | Retain | Released | kcna-lab18/ |
| archive-claim | tracklane-manual | | <unset> | | 17m | |

STATUS: Released, not Available and not deleted. The data is still on the node. This is exactly what a seven-year customs retention requirement needs: deleting a namespace or a claim must not destroy the archive.

A Released PV cannot be re-bound. Prove it:

```
kubectl apply -f manifests/02-static-pvc.yaml
sleep 10
kubectl -n kcna-lab18 get pvc archive-claim
```

```
persistentvolumeclaim/archive-claim created
NAME          STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
VOLUMEATTRIBUTESCLASS  AGE
archive-claim  Pending                                tracklane-manual  <unset>
10s
```

The recreated claim has a new UID; the PV's claimRef.uid still points at the old one, so the controller refuses. To reclaim it an administrator must deliberately clear the stale reference:

```
kubectl patch pv tracklane-archive-pv --type=json \
-p='[{"op":"remove","path":"/spec/claimRef/uid"}, {"op":"remove","path":"/spec/claimRef/resourceVersion"}]'
```

```
persistentvolume/tracklane-archive-pv patched
```

```
sleep 10
kubectl -n kcna-lab18 get pvc archive-claim
```

| NAME | STATUS | VOLUME | CAPACITY | ACCESS MODES | STORAGECLASS |
|---------------|--------|----------------------|----------|--------------|------------------|
| | | AGE | | | |
| archive-claim | Bound | tracklane-archive-pv | 1Gi | RWO | tracklane-manual |
| <unset> | | 33s | | | |

Re-bound. Now prove the **data** survived the whole release-and-readmit cycle by bringing the auditor back:

```
kubectl apply -f manifests/04-archive-reader.yaml
kubectl -n kcna-lab18 wait --for=condition=Ready pod/archive-auditor --timeout=120s
kubectl -n kcna-lab18 logs archive-auditor
```

```
pod/archive-auditor created
pod/archive-auditor condition met
--- provenance written by the (now deleted) seeder Pod ---
sealed_by=archive-seeder
sealed_utc=2026-09-05T05:02:41Z
archive_rows=30
retention_class=gold
--- archive row count as read back ---
rows_read=30
--- can this auditor modify the archive? ---
read-only mount refused the write (expected)
```

sealed_utc is the original seal time from Step 5, unchanged. The claim was deleted, the volume was Released, an administrator re-admitted it, and not one byte moved. That two-step - release, then a human deliberately re-admits the volume - is the entire value of Retain.

If the patch reports remove operation does not apply: doc is missing path, the field was already absent; the operation is idempotent in effect. Drop the missing path from the JSON patch and re-run.

Contrast with Delete: had you deleted render-cache's PVC, the local-path provisioner would have removed the directory and the PV, and the bytes would be gone with no recovery path.

5. Verification

Run these checks after Step 12, with archive-auditor and deployment/render-cache both running.

```
bash verification/checks.sh
```

```
== Lab 18 verification - namespace kcna-lab18 ==
[PASS] namespace kcna-lab18 exists
[PASS] PersistentVolume tracklane-archive-pv exists and is cluster-scoped
[PASS] tracklane-archive-pv capacity=1Gi accessMode=ReadWriteOnce reclaim=Retain
[PASS] tracklane-archive-pv is pre-bound via claimRef to kcna-lab18/archive-claim
[PASS] pvc/archive-claim is Bound to tracklane-archive-pv
[PASS] archive-claim requested 512Mi but was granted the PV's full 1Gi
[PASS] the archive on the PV holds 30 rows (matches data/manifest-archive.csv)
[PASS] PROVENANCE on the PV was written by a Pod that no longer exists
[PASS] storageclass tracklane-retain exists with reclaimPolicy=Retain
[PASS] tracklane-retain uses volumeBindingMode=WaitForFirstConsumer
[PASS] pvc/render-cache is Bound to a dynamically provisioned pvc-* volume
[PASS] the dynamic PV inherited reclaimPolicy=Delete from storageclass standard
[PASS] deployment/render-cache is available and mounts pvc/render-cache
[PASS] /cache/starts.log survived a rollout restart (2+ start records)
-----
14 passed, 0 failed
```

Full transcript: verification/expected-output.md.

6. Failure injection

The ticket: **"PLAT-4471 - archive PVC Pending for 20 minutes, blocking the compliance cutover."**

```
kubectl apply -f manifests/08-pvc-typo-broken.yaml
```

```
persistentvolumeclaim/archive-claim-typo created
```

```
kubectl -n kcna-lab18 get pvc
```

| NAME | STATUS | VOLUME | CAPACITY | ACCESS |
|--------------------|---------|--|----------|--------|
| archive-claim | Bound | tracklane-archive-pv | 1Gi | RW0 |
| tracklane-manual | <unset> | 21m | | |
| archive-claim-typo | Pending | | | |
| tracklane-manul | <unset> | 9s | | |
| render-cache | Bound | pvc-8b2e17d4-05c9-4f36-a1b7-e39d602c8a54 | 256Mi | RW0 |
| standard | <unset> | 11m | | |

get gives you Pending and nothing else. Always escalate to describe:

```
kubectl -n kcna-lab18 describe pvc archive-claim-typo
```

```
Name:          archive-claim-typo
Namespace:     kcna-lab18
StorageClass:  tracklane-manul
Status:        Pending
Volume:
Labels:        app.kubernetes.io/part-of=tracklane
               kcna.tertiaryinfotech.com/intent=failure-injection
               kcna.tertiaryinfotech.com/lab=18
Annotations:   <none>
Finalizers:    [kubernetes.io/pvc-protection]
Capacity:
Access Modes:
VolumeMode:    Filesystem
Used By:        <none>
Events:
  Type            Reason              Age             From                                     Message
  ----            -
  Warning         ProvisioningFailed  4s (x3 over 18s) persistentvolume-controller storageclass.storage.k8s.io "tracklane-manul" not found
```

Diagnosis, step by step:

1. Type: Warning - compare with Step 9's Normal / WaitForFirstConsumer. This one is a genuine fault.
2. Reason: ProvisioningFailed from persistentvolume-controller.
3. The message is unambiguous: storageclass.storage.k8s.io "tracklane-manul" not found.
4. Confirm against reality:

```
kubectl get storageclass
```

| NAME | PROVISIONER | RECLAIMPOLICY | VOLUMEBINDINGMODE |
|--------------------|-----------------------|---------------|----------------------|
| standard (default) | rancher.io/local-path | Delete | WaitForFirstConsumer |
| tracklane-retain | rancher.io/local-path | Retain | WaitForFirstConsumer |

tracklane-manul is not there. tracklane-manual is not there either - because for **static** provisioning the class name is just a label used for matching, and no StorageClass object needs to exist for it. Which raises the subtle question:

Why did archive-claim (class tracklane-manual, also not a real StorageClass) bind, while archive-claim-typo (class tracklane-manul) failed?

>

Because a PV exists with storageClassName: tracklane-manual. The controller's algorithm is: look for an unbound PV whose class, access modes, volumeMode and capacity satisfy the claim; if none exists and a StorageClass of that name exists, ask its provisioner. tracklane-manul matches no PV **and** names no StorageClass, so both paths fail and the controller reports the second failure - the missing class. One-character typos in storageClassName are the single most common cause of an indefinitely Pending PVC.

Fix and confirm (do not edit the broken file; a PVC's storageClassName is immutable, so you must replace the object):

```
kubectl -n kcna-lab18 delete pvc archive-claim-typo
```

```
persistentvolumeclaim "archive-claim-typo" deleted
```

Prove the immutability claim for yourself first, if you like:

```
kubectl -n kcna-lab18 patch pvc archive-claim -p '{"spec":{"storageClassName":"standard"}}'
```

The PersistentVolumeClaim "archive-claim" is invalid: spec: Forbidden: spec is immutable after creation except resources.requests and volumeAttributesClassName for bound claims

A second, more subtle Pending to recognise. A claim can also hang because **nothing consumes it**. Compare the two events side by side - this is the single most useful diagnostic habit in Kubernetes storage:

```
kubectl -n kcna-lab18 get events --field-selector involvedObject.kind=PersistentVolumeClaim \
--sort-by=.lastTimestamp -o custom-columns='TYPE:.type,REASON:.reason,OBJECT:.involvedObject.name,MESSAGE:.message'
```

| TYPE | REASON | OBJECT | MESSAGE |
|---------|-----------------------|--------------------|--|
| Normal | WaitForFirstConsumer | render-cache | waiting for first consumer to be created before binding |
| Normal | ProvisioningSucceeded | render-cache | Successfully provisioned volume pvc-8b2e17d4-05c9-4f36-a1b7-e39d602c8a54 |
| Warning | ProvisioningFailed | archive-claim-typo | storageclass.storage.k8s.io "tracklane-manul" not found |

Normal = wait. Warning = act.

7. Troubleshooting

| Symptom | Likely cause | Diagnostic command | Fix | |
|---|---|--|--|--|
| PVC Pending, event Normal / WaitForFirstConsumer | The StorageClass uses volumeBindingMode: WaitForFirstConsumer and no Pod consumes the claim yet | <code>`kubectl -n kcna-lab18 describe pvc <name> \</code>                            | <code>tail -6`</code> | Nothing is broken. Create the Pod/Deployment that mounts the claim |
| PVC Pending, event Warning / ProvisioningFailed ... storageclass ... not found | storageClassName typo, or the class was never created | <code>kubectl get storageclass</code> | Delete and recreate the PVC with the correct class - storageClassName is immutable | |
| PVC Pending with no events at all | Static provisioning: no PV matches on class, capacity, access mode or volumeMode | <code>kubectl get pv then compare each field</code> | Create a matching PV, or point the claim at a real StorageClass | |
| PVC Pending, a matching PV exists but stays Released | The PV's claimRef.uid still names a deleted claim | <code>kubectl get pv <pv> -o jsonpath='{.spec.claimRef}'</code> | Clear claimRef.uid (and resourceVersion) with a JSON patch, as in Step 12 | |
| Pod ContainerCreating, event Multi-Attach error for volume ... Volume is already exclusively attached to one node | An RWO volume is being pulled to a second node during a rolling update | <code>kubectl -n kcna-lab18 describe pod <pod></code> | Set strategy.type: Recreate, or move to an RWX-capable driver | |
| PVC deleted but stuck Terminating | The kubernetes.io/pvc-protection finalizer: a Pod still references the claim | <code>`kubectl -n kcna-lab18 describe pvc <name> \</code>                            | <code>grep -A2 Used`</code> | Delete the consuming Pods. Do not force-remove the finalizer - that orphans the volume |
| <code>kubectl delete pvc</code> destroyed the data | The StorageClass reclaim policy is Delete | <code>kubectl get pv <pv> -o jsonpath='{.spec.persistentVolumeReclaimPolicy}'</code> | Use a Retain class for anything with a retention obligation. There is no undo | |
| PV shows Available but a same-class claim will not bind | Access mode or volumeMode mismatch, or the PV's claimRef reserves it for another claim | <code>kubectl describe pv <pv></code> | Align accessModes / volumeMode, or target the reserved claim name exactly | |

8. Cleanup

This lab created two cluster-scoped objects, which a namespace delete cannot remove. Delete them by explicit name - never with a label selector or an unscoped `kubectl delete pv --all`.

Order matters. Delete the namespace first so the claims release their volumes, then the PV, then the StorageClass.

```
kubectl delete namespace kcna-lab18 --wait=true
```

```
namespace "kcna-lab18" deleted
```

The standard-class PV for render-cache is reclaimed automatically because its policy is Delete:

```
kubectl get pv | grep kcna-lab18 || echo "no dynamically provisioned volumes remain"
```

```
tracklane-archive-pv 1Gi RWO Retain Released kcna-lab18/archive-claim tracklane-
manual <unset> 38m
```

Only the Retain volume survives - precisely as designed. Remove it by name:

```
kubectl delete persistentvolume tracklane-archive-pv
```

```
persistentvolume "tracklane-archive-pv" deleted
```

```
kubectl delete storageclass tracklane-retain
```

```
storageclass.storage.k8s.io "tracklane-retain" deleted
```

Confirm the cluster is back to its original storage configuration:

```
kubectl get storageclass
kubectl get pv
kubectl get namespace kcna-lab18
```

| NAME | PROVISIONER | RECLAIMPOLICY | VOLUMEBINDINGMODE |
|---|-----------------------|---------------|----------------------------|
| ALLOWVOLUMEEXPANSION | AGE | | |
| standard (default) | rancher.io/local-path | Delete | WaitForFirstConsumer false |
| 81m | | | |
| No resources found | | | |
| Error from server (NotFound): namespaces "kcna-lab18" not found | | | |

The directory /tmp/tracklane-archive still exists inside the kind node container. Deleting a Retain PV releases the Kubernetes object but never touches the backing store - reclaiming the actual bytes is a deliberate administrative act, and on this cluster it happens when you kind delete cluster.

9. What you learned

- * PV and PVC split supply from demand. A PersistentVolume is cluster-scoped and belongs to whoever runs the cluster; a PersistentVolumeClaim is namespaced and belongs to the workload. The Pod spec only ever names the claim, which is why the same Deployment YAML runs on kind, EKS and on-prem Ceph unchanged.
- * Binding is one-to-one, "at least as large as requested", and permanent. The controller records the claim's UID in spec.claimRef.uid; a recreated claim with the same name gets a new UID and will not re-bind.
- * Access modes are a property of the driver, not a wish. RWO means one *node*, not one *Pod* - ReadWriteOncePod (stable v1.29) is the mode that means one Pod. On single-node kind with local-path, RWO is all you can honestly have.
- * Reclaim policy is a data-retention decision, not a technical detail. Delete destroys the backing store with the claim; Retain parks the PV in Released and demands a human to re-admit it. Map it from your retention policy, as data/retention-policy.csv does.

* A StorageClass is a named profile over a provisioner - driver, parameters, reclaim policy, binding mode, expansion. Two classes can share one driver and differ only in lifecycle, which is exactly what tracklane-retain demonstrates.

* WaitForFirstConsumer produces a Pending PVC on purpose. Normal / WaitForFirstConsumer means wait; Warning / ProvisioningFailed means act. Learning to read that distinction from describe pvc is the single highest-value storage debugging skill.

* CSI is the plugin boundary. In-tree drivers are gone; every real driver is out-of-tree and speaks the CSI gRPC contract. kind's local-path-provisioner is not CSI, which is why kubectl get csidrivers is empty - a useful reminder that this cluster teaches the API, not production durability.

Carry into Lab 19: a Deployment shares one PVC across all its replicas. A StatefulSet cannot work that way - each replica needs *its own* volume, created automatically and keyed to its ordinal. That is volumeClaimTemplates.

10. Further reading

* Kubernetes documentation - *Persistent Volumes* (the authoritative reference for phases, binding, reclaiming and access modes): <<https://kubernetes.io/docs/concepts/storage/persistent-volumes/>>

* Kubernetes documentation - *Storage Classes*: <<https://kubernetes.io/docs/concepts/storage/storage-classes/>>

* Kubernetes documentation - *Dynamic Volume Provisioning*:
<<https://kubernetes.io/docs/concepts/storage/dynamic-provisioning/>>

* Kubernetes documentation - *Volume Binding Mode*:
<<https://kubernetes.io/docs/concepts/storage/storage-classes/#volume-binding-mode>>

* Kubernetes blog - *Kubernetes v1.29: ReadWriteOncePod access mode goes GA*:
<<https://kubernetes.io/blog/2023/12/18/read-write-once-pod-access-mode-ga/>>

* Container Storage Interface specification:
<<https://github.com/container-storage-interface/spec/blob/master/spec.md>>

* kind documentation - *Persistent Volumes* and the bundled local-path-provisioner:
<<https://kind.sigs.k8s.io/docs/user/local-registry/>> and <<https://github.com/rancher/local-path-provisioner>>

* CNCF KCNA Curriculum - *Container Orchestration → Storage*: <<https://github.com/cncf/curriculum>>